

# SIMATS ENGINEERING

## ASSESSMENT TOOL 3

**COURSE CODE: MLA03**

**COURSE NAME: Reinforcement learning for Environment and Agent**

**COURSE OUTCOME COVERED**

*CO5: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

*Assessment Tool Weightage: 30%*

**Dr DUMPALA PRASANTH**

**QUESTIONS: 10    Total Marks: 50**

**Annexure A**

### Analytical Problem-Solving Assessment

**Duration: 1hr**

---

**1.** A warehouse robot must determine the shortest path to transport goods while avoiding obstacles. Apply **Dynamic Programming** to develop an optimal policy and explain how value iteration can improve navigation efficiency. **( 5 Marks)**

**Answer: (5 Marks)**

#### ***1. MDP Formulation***

The warehouse floor is modelled as a finite Markov Decision Process (MDP) defined by the tuple (S, A, P, R,  $\gamma$ ):

- S: grid cells representing robot positions (obstacle cells are terminal/forbidden states).
- A: {up, down, left, right} movement actions.
- $P(s'|s,a)$ : transition probability to the next cell (deterministic on success, 0 if it leads into an obstacle).
- $R(s,a,s')$ : -1 for every move (to encourage shortest path), large negative reward for hitting an obstacle, +large reward on reaching the goal shelf.
- $\gamma$ : discount factor ( $0 < \gamma \leq 1$ ) that weighs future rewards.

#### ***2. Bellman Optimality Equation***

$$V^*(s) = \max_a \sum_{s'} P(s'|s,a) [ R(s,a,s') + \gamma V^*(s') ]$$

Dynamic Programming exploits the full knowledge of the environment model (grid layout, obstacle positions) to solve this equation recursively using either Policy Iteration or Value Iteration.

### 3. Value Iteration Algorithm

```
Initialize V(s) = 0 for all states s
repeat:
    delta = 0
    for each state s in S:
        v_old = V(s)
        V(s) = max_a sum_s' P(s'|s,a) [ R(s,a,s') + gamma * V(s') ]
        delta = max(delta, |v_old - V(s)|)
until delta < theta      # convergence threshold

# Extract optimal policy
for each state s:
    pi*(s) = argmax_a sum_s' P(s'|s,a) [ R(s,a,s') + gamma * V(s') ]
```

### 4. Application to the Warehouse Robot

Value iteration repeatedly sweeps every grid cell, back-propagating the goal's positive reward and the obstacles' penalty until the value function converges. Cells closer to obstacles receive lower values, while cells that lie on efficient routes to the goal receive higher values. The greedy policy extracted from  $V^*(s)$  always moves the robot toward the neighbouring cell with the highest value, which corresponds to the shortest, obstacle-free path.

### 5. How It Improves Navigation Efficiency

- Guarantees convergence to the globally optimal policy because it is based on the contraction property of the Bellman operator.
- Avoids exhaustive path search (e.g., brute-force enumeration) - complexity is  $O(|S|^2|A|)$  per sweep instead of exponential path enumeration.
- Automatically re-plans if the reward/obstacle map changes, since the value function can be recomputed.
- Produces a stationary optimal policy usable in real time - the robot only performs an argmax lookup at every step, so online decision making is fast.

2. A recommendation system collects customer interactions over multiple sessions. Apply **Monte Carlo Control** to update the policy and explain how the agent improves recommendations with experience. ( 5 Marks)

**Answer: (5 Marks)**

#### 1. Problem Setup

Each customer session is treated as an episode: state = customer/session context (browsing history, cart items), action = item recommended, reward = click/purchase signal, episode ends at session close.

Because Monte Carlo (MC) methods learn purely from complete episodes without needing a transition model, they suit recommendation logs collected session by session.

## 2. Monte Carlo Control (On-policy, $\epsilon$ -greedy, first-visit)

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{(T-t-1)} R_T \quad (\text{return from time } t)$$

$$Q(s,a) \leftarrow Q(s,a) + (1/N(s,a)) [G_t - Q(s,a)] \quad (\text{incremental sample-average update})$$

## 3. Algorithm

```
Initialize Q(s,a) arbitrarily, N(s,a) = 0, epsilon-greedy policy pi
repeat for each session (episode):
    generate episode s0,a0,r1,s1,a1,r2,...,sT using pi
    G = 0
    for t = T-1 down to 0:
        G = gamma * G + r_(t+1)
        if (s_t, a_t) is first occurrence in episode:
            N(s_t,a_t) += 1
            Q(s_t,a_t) += (1/N(s_t,a_t)) * (G - Q(s_t,a_t))
            # policy improvement: epsilon-greedy over Q
            A* = argmax_a Q(s_t,a)
            pi(a|s_t) = 1 - epsilon + epsilon/|A|    if a == A*
            pi(a|s_t) = epsilon/|A|                otherwise
```

## 4. How the Agent Improves with Experience

- Every completed session contributes a full return  $G_t$ , giving an unbiased estimate of the true action-value, unlike bootstrapped methods.
- As more sessions are observed, the sample-average  $Q(s,a)$  converges (Law of Large Numbers) to the true expected reward of recommending item  $a$  in context  $s$ .
- The  $\epsilon$ -greedy policy improvement step (Generalized Policy Iteration) ensures the recommender keeps exploring under-trying items while progressively favouring items with proven high engagement.
- Over many customers and sessions, GLIE (Greedy in the Limit with Infinite Exploration) conditions guarantee the policy converges to the optimal recommendation strategy.

3. A delivery robot operates in an uncertain environment. Apply the **SARSA** algorithm to update the action-value function and explain how the learned policy adapts to environmental changes. ( 5 Marks)

### Answer: (5 Marks)

#### 1. Why SARSA

SARSA (State-Action-Reward-State-Action) is an on-policy Temporal-Difference (TD) control method. Because the delivery robot must learn while acting in a real, uncertain environment (traffic, weather, road closures), SARSA is preferred over pure MC methods since it updates after every single step instead of waiting for episode completion, and it evaluates the policy it is actually following, which keeps behaviour safer during learning.

#### 2. Update Rule

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

where  $a'$  is the action actually chosen in the next state  $s'$  by the current (e.g.,  $\epsilon$ -greedy) policy - hence the name SARSA ( $s, a, r, s', a'$ ).

### 3. Algorithm

```
Initialize  $Q(s,a)$  arbitrarily for all  $s,a$ 
for each episode (delivery run):
    initialize  $s$ 
    choose  $a$  from  $s$  using policy derived from  $Q$  ( $\epsilon$ -greedy)
    repeat for each step of the episode:
        take action  $a$ , observe reward  $r$  and next state  $s'$ 
        choose  $a'$  from  $s'$  using policy derived from  $Q$  ( $\epsilon$ -greedy)
         $Q(s,a) \leftarrow Q(s,a) + \alpha * (r + \gamma * Q(s',a') - Q(s,a))$ 
         $s \leftarrow s'$  ;  $a \leftarrow a'$ 
    until  $s$  is terminal (delivery complete)
```

### 4. Adapting to Environmental Changes

- Because updates happen online at every step, the robot immediately reduces the value of an action that leads to an unexpected obstacle (e.g., a blocked road), instead of waiting for a full episode as MC would.
- Being on-policy,  $Q$  converges to the value of the policy actually being executed (including its exploratory  $\epsilon$ -greedy behaviour), so the learned path avoids risky shortcuts near hazards - important for a physical robot operating under uncertainty.
- Continual learning across delivery runs lets  $Q(s,a)$  track a non-stationary environment: as traffic patterns or obstacles change over time,  $Q$  values are continuously corrected, so the greedy policy  $\pi(s)=\text{argmax}_a Q(s,a)$  automatically re-routes the robot.

---

4. A robotic arm performs continuous object manipulation. Apply the **REINFORCE** algorithm to develop a policy that maximizes task completion accuracy. ( 5 Marks)

**Answer: (5 Marks)**

#### 1. Why a Policy-Gradient Method

The robotic arm's joint torques/angles form a continuous action space, so tabular value-based methods (Q-learning, SARSA) are not directly applicable. REINFORCE, a Monte-Carlo policy-gradient algorithm, directly parameterizes and optimizes a stochastic policy  $\pi\theta(a|s)$ , typically a Gaussian policy over continuous actions, making it well suited to continuous manipulation tasks.

#### 2. Policy Gradient Theorem

$$\nabla_{\theta} J(\theta) = E \pi_{\theta} [ \nabla_{\theta} \log \pi_{\theta}(a|s) \cdot G_t ]$$

For continuous actions the policy is commonly modelled as a Gaussian:

$$\pi_{\theta}(a|s) = N(\mu_{\theta}(s), \sigma_{\theta}(s)^2)$$

#### 3. REINFORCE Update

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \cdot G_t$$

A baseline  $b(s)$  (e.g., a learned state-value estimate) is usually subtracted from  $G_t$  to reduce variance without introducing bias:

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot (G_t - b(s_t))$$

#### 4. Algorithm

```
Initialize policy parameters theta
repeat for each episode (manipulation attempt):
    generate trajectory s0,a0,r1,...,sT using pi_theta(.|s)
    for t = 0 to T-1:
        G_t = sum_{k=t}^{T-1} gamma^{k-t} * r_{k+1}
        theta <- theta + alpha * grad_theta log(pi_theta(a_t|s_t)) * (G_t - b(s_t))
```

#### 5. Maximizing Task-Completion Accuracy

- The reward function is shaped for the manipulation task (e.g., negative distance of end-effector to target + large bonus for a successful grasp/placement), so gradient ascent on  $J(\theta)$  directly increases expected task success.
- The stochastic Gaussian policy naturally provides exploration in continuous joint-torque space, letting the arm discover smoother, more accurate trajectories over many trials.
- As  $\theta$  is updated across episodes,  $\sigma_{\theta}(s)$  typically shrinks, so the policy becomes progressively more precise (less random) once it has found accurate manipulation strategies, improving completion accuracy over time.

5. An autonomous vehicle is trained using **Q-Learning** and **Deep Q-Networks (DQN)**. Analyze the differences in learning performance, convergence speed, and scalability for complex driving environments. ( 5 Marks)

#### Answer: (5 Marks)

##### 1. Q-Learning (Tabular, off-policy TD control)

$$Q(s,a) \leftarrow Q(s,a) + \alpha [ r + \gamma \max_{a'} Q(s',a') - Q(s,a) ]$$

Q-learning stores one value per (state, action) pair in a table. This works only when the state space is small and discrete.

##### 2. Deep Q-Network (DQN)

DQN replaces the table with a neural network  $Q(s,a;w)$  that generalizes across the vehicle's high-dimensional, continuous sensor state (camera frames, LiDAR point clouds, speed, lane position). It is trained by minimizing the TD loss:

$$L(w) = E[ ( r + \gamma \max_{a'} Q(s',a';w^-) - Q(s,a;w) )^2 ]$$

using two key stabilizers: an experience replay buffer (breaks correlation between consecutive driving frames) and a slowly-updated target network with weights  $w^-$  (reduces moving-target instability).

##### 3. Comparative Analysis

| Aspect | Q-Learning (tabular) | DQN | Implication for Driving |
|--------|----------------------|-----|-------------------------|
|--------|----------------------|-----|-------------------------|

|                      |                                                                         |                                                                                                                     |                                                                 |
|----------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Learning performance | Exact but limited to small/discrete states                              | Generalizes over raw high-dimensional sensor input                                                                  | DQN needed for realistic driving perception                     |
| Convergence speed    | Guaranteed convergence (with GLIE) but very slow for large state spaces | Faster in practice for large states, but no formal convergence guarantee; can be unstable without replay/target net | DQN trains feasibly on continuous driving states                |
| Scalability          | Poor - table size grows exponentially with state features               | Scales well via function approximation and weight sharing                                                           | DQN is the only viable option for full-scale autonomous driving |

#### 4. Conclusion

Tabular Q-learning is simple and provably convergent but cannot handle the continuous, high-dimensional perception space of an autonomous vehicle. DQN trades formal convergence guarantees for scalability and generalization, making it the practical choice for complex, real-world driving environments.

---

6. A game-playing AI is implemented using **DQN**, **Double DQN (DDQN)**, and **Dueling DQN**. Analyze the advantages of DDQN and Dueling DQN over the standard DQN architecture. **(5 Marks)**

**Answer: (5 Marks)**

##### 1. Limitation of Standard DQN: Overestimation Bias

Standard DQN uses the same network to both select and evaluate the best next action:

$$y = r + \gamma \max_{a'} Q(s', a'; w^-)$$

Because max is applied over noisy estimates, Q-values are systematically overestimated, which can mislead the game-playing agent toward sub-optimal actions.

##### 2. Double DQN (DDQN)

DDQN decouples action selection from action evaluation by using the online network to choose the action and the target network to evaluate it:

$$y = r + \gamma \cdot Q(s', \operatorname{argmax}_{a'} Q(s', a'; w) ; w^-)$$

Advantage: this removes most of the positive bias, giving more accurate value estimates and a more stable, often higher-scoring policy in games such as Atari.

##### 3. Dueling DQN

Dueling DQN changes the network architecture, splitting the final layers into two streams: a state-value stream  $V(s)$  and an action-advantage stream  $A(s, a)$ , which are combined as:

$$Q(s, a) = V(s) + (A(s, a) - (1/|A|) \sum_{a'} A(s, a'))$$

Advantage: in many game states the choice of action barely affects the outcome (e.g., idle frames), so learning  $V(s)$  separately lets the agent evaluate state quality without needing to learn the effect of every

action - this speeds up learning and improves policy evaluation in states with many similar-valued actions.

#### **4. Combined Benefit over Standard DQN**

- More accurate Q-value estimates (DDQN) → fewer sub-optimal moves caused by overestimated actions.
  - Better generalization across actions within a state (Dueling) → faster learning, especially in large action spaces typical of complex games.
  - Both variants are drop-in improvements that keep DQN's replay buffer and target network, so they combine easily (Dueling-DDQN) for further gains in stability and final score.
- 

7. A smart drone navigation system uses **A2C** and **A3C** algorithms. Analyze the impact of parallel learning on convergence speed, computational efficiency, and policy stability. **( 5 Marks)**

#### **Answer: (5 Marks)**

##### **1. Actor-Critic Foundation**

Both algorithms combine an actor (policy  $\pi_\theta(a|s)$ ) and a critic (value function  $V_\phi(s)$ ) that reduces the variance of the policy gradient by using the advantage function:

$$A(s,a) = r + \gamma V_\phi(s') - V_\phi(s)$$
$$\nabla_\theta J(\theta) = E[ \nabla_\theta \log \pi_\theta(a|s) \cdot A(s,a) ]$$

##### **2. A3C - Asynchronous Advantage Actor-Critic**

Multiple drone-simulation workers run in parallel threads, each with its own copy of the environment and network. Each worker computes gradients independently and asynchronously pushes updates to a shared global network, then pulls the latest parameters back.

##### **3. A2C - Advantage Actor-Critic (synchronous)**

A2C runs the same parallel workers, but waits for all of them to finish a batch of steps, averages their gradients, and performs a single synchronous update to the global network - removing the asynchrony of A3C while keeping the parallel data collection.

##### **4. Impact of Parallelism**

- Convergence speed: parallel workers explore different parts of the drone's flight-state space simultaneously (varied wind, obstacles, altitudes), decorrelating the training data in a way similar to experience replay, which speeds up convergence relative to a single-agent actor-critic.
- Computational efficiency: multiple CPU/GPU workers fully utilize available hardware; A3C's asynchronous updates avoid the synchronization wait time, giving higher throughput, whereas A2C trades some speed for simpler, more efficient batched GPU computation.
- Policy stability: A3C's asynchronous, uncoordinated updates can introduce stale gradients (a worker updates using parameters that have since changed), causing noisier, less stable learning.

A2C's synchronous averaging removes stale gradients, yielding more stable and reproducible convergence - an important property for safety-critical drone navigation.

## 5. Conclusion

Parallel learning substantially accelerates training for drone navigation by providing diverse, decorrelated experience. A3C favours raw computational throughput, while A2C favours stability and reproducibility, which is often preferable when the learned policy must be verified before deployment on real drones.

---

8. A healthcare decision-support system operates with incomplete patient information. Analyze how a Partially Observable Markov Decision Process (POMDP) improves decision-making compared to a standard MDP. (5 Marks)

**Answer: (5 Marks)**

### 1. Limitation of the Standard MDP

A standard MDP assumes the agent observes the true state  $s$  directly. In healthcare, the true patient state (underlying disease progression) is never fully known - only noisy or partial observations are available (symptoms, test results), violating the MDP's full-observability assumption and leading to poor decisions if an MDP policy is applied naively to raw observations.

### 2. POMDP Formulation

A POMDP extends the MDP to the tuple  $(S, A, P, R, \Omega, O, \gamma)$ , adding:

- $\Omega$ : the set of possible observations (test results, reported symptoms).
- $O(o|s',a)$ : probability of observing  $o$  after taking action  $a$  and landing in state  $s'$ .

### 3. Belief State

Because the true state is hidden, the agent maintains a belief  $b(s)$  - a probability distribution over possible patient states - updated after every new observation via Bayes' rule:

$$b'(s') = \eta \cdot O(o|s',a) \cdot \sum_s P(s'|s,a) b(s)$$

where  $\eta$  is a normalizing constant. The POMDP is then solved as an MDP over the continuous belief-state space, choosing actions to maximize expected value with respect to  $b$ :

$$V^*(b) = \max_a [ \sum_s b(s) R(s,a) + \gamma \sum_o P(o|b,a) V^*(b') ]$$

### 4. Why This Improves Healthcare Decision-Making

- The belief state formally aggregates all uncertainty from incomplete labs/symptoms into a single probabilistic summary, rather than forcing a brittle single-state assumption as an MDP would.
- POMDP policies can choose information-gathering actions (e.g., order an additional test) purely to reduce uncertainty before committing to a risky treatment - a behaviour an MDP cannot represent since it assumes the state is already known.



- Decisions are robust to diagnostic ambiguity: the recommended treatment reflects a weighted average over plausible patient conditions, reducing the risk of a confidently wrong action based on incomplete data.
- As new observations (test results) arrive, the belief is updated continuously, allowing the system to refine its recommendation dynamically as more patient information becomes available.

9. A robotic navigation system is trained using **PPO**, **TRPO**, and **DDPG**. Evaluate the suitability of each algorithm for continuous control tasks based on stability, sample efficiency, and computational complexity. ( 5 Marks)

**Answer: (5 Marks)**

### 1. TRPO - Trust Region Policy Optimization

TRPO constrains each policy update to stay within a “trust region” measured by KL-divergence, guaranteeing monotonic policy improvement:

$$\text{maximize } E[ \pi\theta(a|s)/\pi\theta_{old}(a|s) \cdot A(s,a) ] \quad \text{subject to} \quad E[ KL(\pi\theta_{old} || \pi\theta) ] \leq \delta$$

This constrained optimization requires computing a Fisher-Information matrix and conjugate-gradient steps, which is computationally expensive.

### 2. PPO - Proximal Policy Optimization

PPO approximates TRPO's trust region with a simple clipped surrogate objective, avoiding the second-order optimization:

$$L(\theta) = E[ \min( r\theta(a|s) \cdot A(s,a), \text{clip}(r\theta(a|s), 1-\epsilon, 1+\epsilon) \cdot A(s,a) ) ], \quad \text{where } r\theta = \pi\theta(a|s)/\pi\theta_{old}(a|s)$$

### 3. DDPG - Deep Deterministic Policy Gradient

DDPG is an off-policy actor-critic method for continuous action spaces that learns a deterministic policy  $\mu\theta(s)$  and a critic  $Q(s,a;w)$ , updated via the deterministic policy gradient, and uses a replay buffer plus target networks:

$$\nabla_{\theta} J \approx E[ \nabla_a Q(s,a;w)|_{a=\mu\theta(s)} \cdot \nabla_{\theta} \mu\theta(s) ]$$

### 4. Comparative Evaluation for Robotic Navigation

| Criterion                | TRPO                                                 | PPO                                                | DDPG                                                                          |
|--------------------------|------------------------------------------------------|----------------------------------------------------|-------------------------------------------------------------------------------|
| Stability                | Very stable - monotonic improvement guarantee        | Stable in practice via clipping; simpler than TRPO | Less stable - sensitive to hyperparameters and exploration noise              |
| Sample efficiency        | On-policy - moderate, needs fresh data per update    | On-policy - moderate, similar to TRPO              | Off-policy with replay buffer - most sample efficient                         |
| Computational complexity | High - Fisher matrix / conjugate gradient per update | Low - first-order gradient updates only            | Moderate - two networks (actor+critic) with target nets, but simple gradients |

## 5. Suitability Verdict

For a robotic navigation system, PPO offers the best overall balance of stability and low implementation/computational cost and is the most common default choice. TRPO can be used when the strongest stability guarantees are required and compute budget is not a concern. DDPG is preferable when sample efficiency is critical (e.g., learning directly on physical hardware with limited real-world trials), provided its training instability is managed with careful tuning and techniques such as TD3-style improvements.

---

**10.** A smart city traffic management system uses **Multi-Agent Reinforcement Learning (MARL)** to coordinate traffic signals. Evaluate the effectiveness of MARL in improving traffic flow while considering scalability, coordination, fairness, and ethical challenges. **( 5 Marks)**

**Answer: (5 Marks)**

### ***1. MARL Formulation for Traffic Signal Control***

Each traffic-signal intersection is modelled as an agent  $i$  with its own local state  $s_i$  (queue lengths, waiting times), local action  $a_i$  (which phase/light to activate), and reward  $r_i$  (negative of local congestion/waiting time). The joint system is a Markov Game  $(S, A_1..A_n, P, R_1..R_n, \gamma)$ , where the transition depends on the joint action of all agents.

### ***2. Coordination Mechanisms***

- Centralized Training, Decentralized Execution (CTDE): a central critic  $Q_{tot}(s, a_1, ..., a_n)$  is used during training to capture inter-agent effects, while each agent executes its own local policy  $\pi_i$  at run time - keeping execution scalable.
- Value decomposition (e.g., VDN/QMIX-style factorization) approximates the joint action-value as a function of individual agents' values so credit assignment is possible without exhaustive joint-action search:

$$Q_{tot}(s, a) \approx f(Q_1(s_1, a_1), Q_2(s_2, a_2), ..., Q_n(s_n, a_n))$$

- Communication/graph-based coordination lets neighbouring intersections share queue information so a downstream signal can pre-emptively adjust to an upstream green-wave, reducing city-wide congestion rather than just local queues.

### ***3. Effectiveness in Improving Traffic Flow***

Empirically and theoretically, MARL improves on fixed-time or independently-optimized signals because agents learn to anticipate the effect of their phase choice on neighbouring intersections (e.g., forming green corridors), reducing average waiting time and stop-and-go congestion city-wide rather than only at a single junction.

### ***4. Scalability***

As the number of intersections  $n$  grows, the joint action space grows exponentially ( $|A|^n$ ), making fully centralized control intractable. CTDE and value-factorization keep training/execution scalable by

limiting explicit coordination to local neighbourhoods, but very large cities still face partial-observability and non-stationarity (each agent's environment appears to shift as other agents' policies change during learning).

### **5. Fairness and Ethical Challenges**

- Fairness: a purely throughput-maximizing reward can systematically favour high-traffic arterial roads while starving minor roads or pedestrian crossings of green time; reward shaping or explicit fairness constraints are needed to ensure equitable wait times across all road users.
- Safety and accountability: MARL policies are difficult to interpret, raising concerns about accountability when a signal decision contributes to an accident; explainability and human-in-the-loop oversight are required for deployment.
- Emergency vehicles/pedestrians: reward functions must explicitly encode priority for ambulances, fire trucks, and pedestrian safety, since a purely traffic-flow-optimizing agent might not prioritize these correctly.
- Privacy: sensor data (cameras, GPS) used as agent observations must be handled in compliance with data-privacy regulations.

### **6. Conclusion**

MARL is highly effective for city-scale traffic-signal coordination, outperforming independent or fixed-time control by learning cooperative behaviour across intersections. However, its real-world effectiveness depends on addressing scalability through decentralized/factorized architectures, and on carefully designing reward functions and oversight mechanisms to guarantee fairness, safety, and ethical accountability.

---

### ***Code of Conduct:***

*I Charandeep.N.C/192472196 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: Charandeep***

## RUBRICS FOR CALCULATION

| Criteria                 | Weightage | Level 4 (Exemplary)                                                       | Level 3 (Proficient)                                 | Level 2 (Developing)                                 | Level 1 (Beginning)                        |
|--------------------------|-----------|---------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|--------------------------------------------|
| Conceptual Understanding | 25        | Demonstrates deep understanding of concepts with complete clarity         | Explains concepts correctly with minor gaps          | Partial understanding with errors or omissions       | Unable to explain basic concepts           |
| Application              | 25        | Effectively applies concepts to new situations; solves problems logically | Applies concepts with minor errors                   | Limited ability to apply concepts; requires guidance | Unable to apply concepts effectively       |
| Analytical Thinking      | 20        | Critically analyzes scenarios with strong justification and reasoning     | Provides reasonable analysis with some justification | Superficial analysis with weak reasoning             | No analytical reasoning demonstrated       |
| Communication            | 15        | Communicates ideas clearly, confidently, and with appropriate terminology | Communicates clearly but with minor hesitation       | Communication lacks clarity or structure             | Unable to communicate ideas effectively    |
| Response to Questions    | 15        | Responds accurately and confidently, extending answers with depth         | Responds correctly but with limited depth            | Responds partially; requires prompting               | Unable to respond to questions effectively |